《人工智能基础及应用》（进阶版）课程

第三章　机器学习项目报告

电商用户近期购买行为分类预测

姓　　名：	倪昊
学　　号：	1406250303
班　　级：	光电 2505
提交日期：	2026 年 10 月 24 日

第三章 机器学习项目报告

1　问题定义

本项目根据电商顾客已有的消费记录，判断其最近 30 天是否购买过商品。输入为年龄、会员类型、累计消费金额、购买件数、平均评分和是否使用折扣；输出为二分类标签：1 表示最近一次购买距今不超过 30 天，0 表示超过 30 天。

原始数据没有直接的“是否购买”字段，而且每位顾客的购买件数都大于 0。因此这里预测的是“近期是否购买”，不是新顾客未来会不会购买。该模型适用于本表中的既有顾客，不能直接用于从未购买的用户。

2　数据及划分

数据文件为 E-commerce Customer Behavior.csv，共 350 条记录、11 个原始字段。标签由 Days Since Last Purchase 是否不超过 30 天生成：正类 226 条，负类 124 条。

程序排除 Customer ID、Gender、City、Satisfaction Level 和 Days Since Last Purchase。其中编号不能代表购买行为，城市和性别并非本实验选择的直接行为特征，满意度可能在购买之后取得，最后一列用于生成标签，必须从输入中去除。会员类型用独热编码转为 3 列，最终输入共有 8 列。

采用分层随机划分，随机种子为 42：训练集 210 条，验证集 70 条，测试集 70 条。只在验证集上选择模型参数，最后才使用测试集评价。数据中除编号外有 180 条完全重复记录，可能使随机划分后的成绩偏高。

3　算法与模型

逻辑回归先把不同特征的数值范围统一，再根据这些特征给出“近 30 天购买过”的预测。程序比较了 5 个 C 值，依据验证集准确率选择 C=0.1。

决策树像一组逐层进行的判断，例如先看某项消费特征是否超过一个数值，再判断下一项。程序比较了最大深度 1～8。树太深可能把训练数据记得过细，遇到新数据时反而表现变差。

主程序分别比较逻辑回归和决策树，用验证集选择参数，再将训练集与验证集合并训练，最后在测试集上评价。另编写四个小程序，依次比较多数类基线、只用年龄、加入累计消费和将树深度从 1 增到 2；每一步都沿用同一次数据划分，便于观察改动的影响。

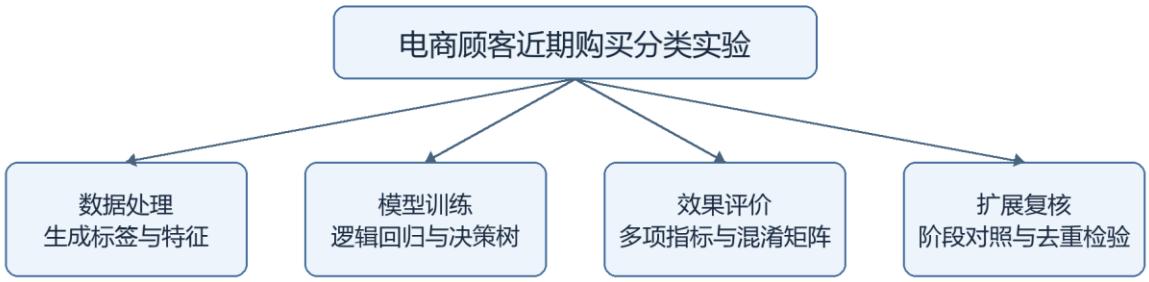


图 1　程序结构图

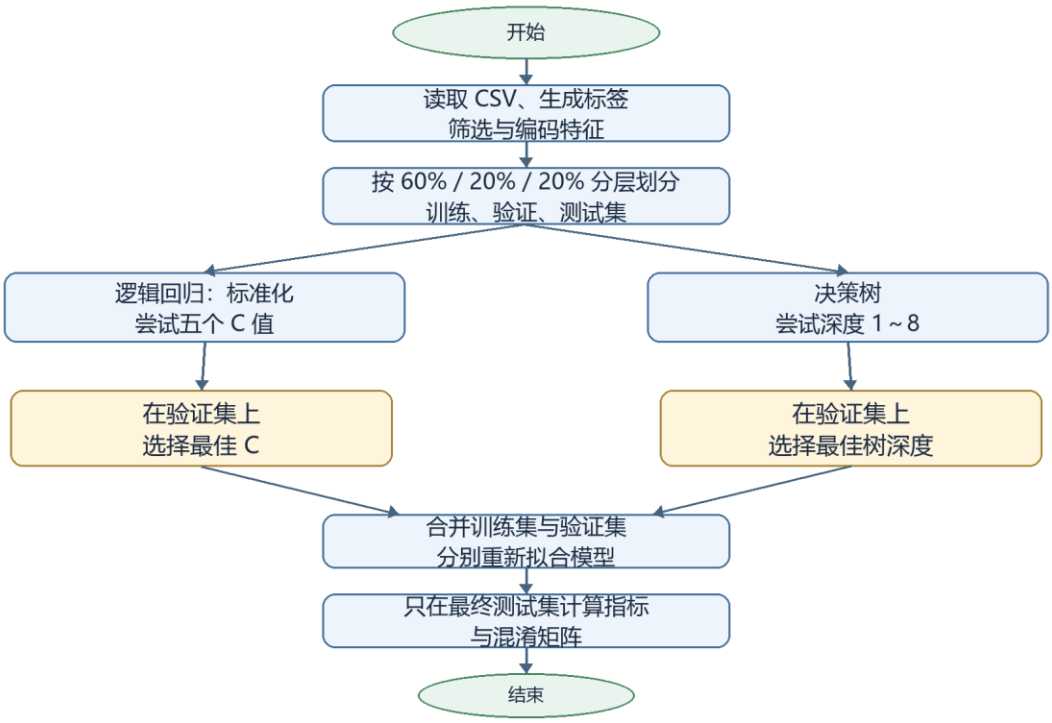


图 2　程序流程图

4　实验结果

4.1　四个阶段的改进

为了看清模型是怎样一步步改进的，我每次只改变一个主要条件。多数类基线总是猜样本较多的“近 30 天购买过”；后面三个阶段使用决策树。准确率表示全部预测中答对的比例，精确率关注预测为“是”的结果有多少是真的，召回率关注实际为“是”的顾客找出了多少。

表 1　四个阶段的实验结果

阶段及改动	验证准确率	测试准确率	精确率	召回率
0　多数类基线	0.643	0.643	0.643	1.000
1　只用年龄	0.786	0.786	0.750	1.000
2　加入累计消费	0.871	0.829	0.946	0.778
3　树深度增至 2	0.943	0.971	0.957	1.000

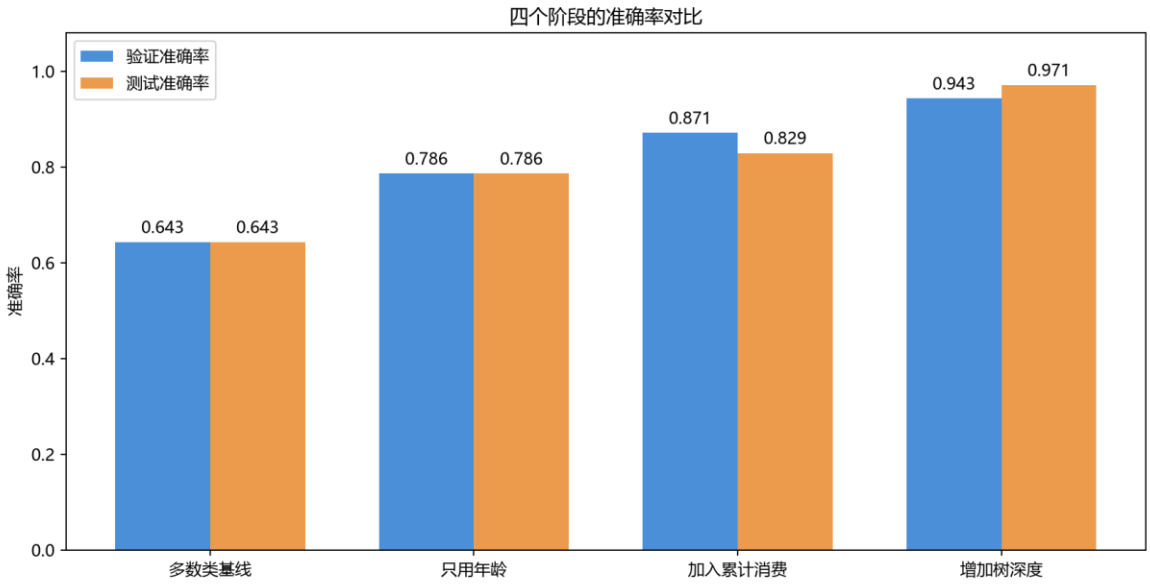


图 3　四个阶段的验证与测试准确率

从阶段 0 到阶段 1，测试准确率由 0.643 提高到 0.786，说明年龄包含一定信息。阶段 2 加入累计消费后，测试准确率提高到 0.829，但召回率从 1.000 降为 0.778，说明只看准确率会漏掉这一变化。阶段 3 保持两个输入特征，把树深度由 1 增到 2，测试准确率达到 0.971，召回率也回到 1.000。这些结果只说明在本次数据划分上改进有效，不能保证换一批顾客也一样。

4.2　最终模型对比

主程序在相同训练、验证、测试划分上比较逻辑回归与决策树。两种模型仅用验证集选择参数，再用训练集和验证集合并训练，最后各在未参与调参的测试集上评价。

表 2　最终模型与多数类基线的结果对比

方法	验证准确率	测试准确率	测试集其他指标
多数类基线	0.643	0.643	精确率 0.643；召回率 1.000
逻辑回归 C=0.1	0.943	0.971	精确率 0.957；召回率 1.000
决策树 深度 2	0.943	0.971	精确率 0.957；召回率 1.000

主程序中的逻辑回归和决策树在测试集上都预测正确 68 条，准确率为 68/70≈0.971；精确率为 45/47≈0.957，召回率为 45/45=1.000。相对多数类基线的 0.643，准确率提高了约 0.328。两种模型这次成绩相同，不能据此说其中一种一定更好。

两张混淆矩阵也相同：实际为“否”的 25 条中，23 条判断正确、2 条误判为“是”；实际为“是”的 45 条全部找出。参数曲线显示，决策树深度超过 4 后训练准确率仍很高，但验证准确率下降；因此本实验选择较浅的深度 2。

```
C:\Users\Lenovo\.conda\envs\pytorch\python.exe E:\ScienceLane\PrimaryAI_Experiment\购买行为分类实验.py
本实验预测的是近 30 天是否购买；排除的列： ['Customer ID', 'Gender', 'City', 'Satisfaction Level', 'Days Since
逻辑回归：准确率=0.971，精确率=0.957，召回率=1.000
决策树：准确率=0.971，精确率=0.957，召回率=1.000
新用户预测：逻辑回归= 0 决策树= 0
选用参数：逻辑回归 C= 0.1 决策树最大深度= 2

进程已结束，退出代码为 0
```

图 4　PyCharm 运行结果截图

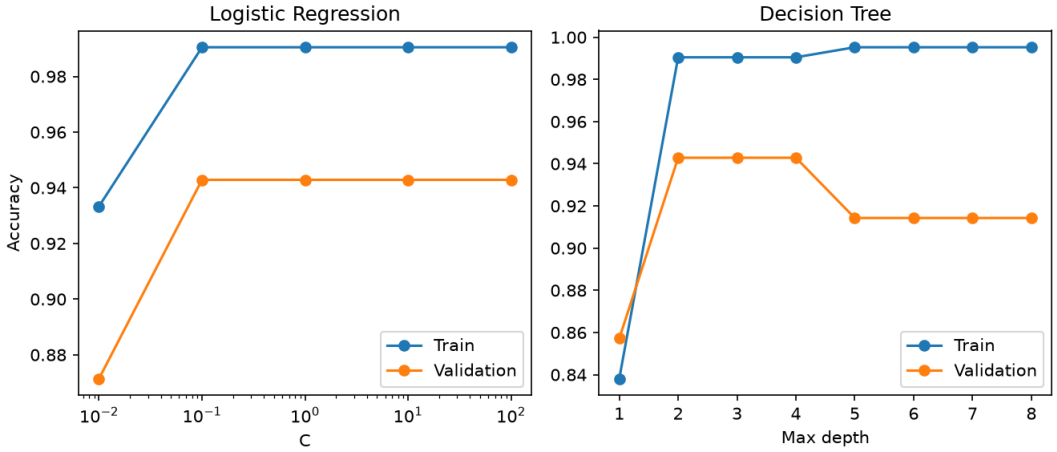


图 5　不同参数下的训练与验证准确率

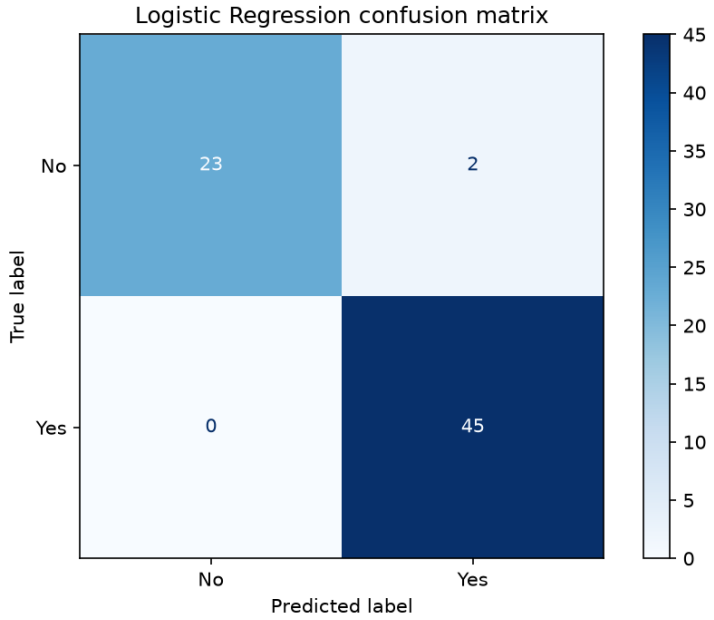


图 6　逻辑回归混淆矩阵

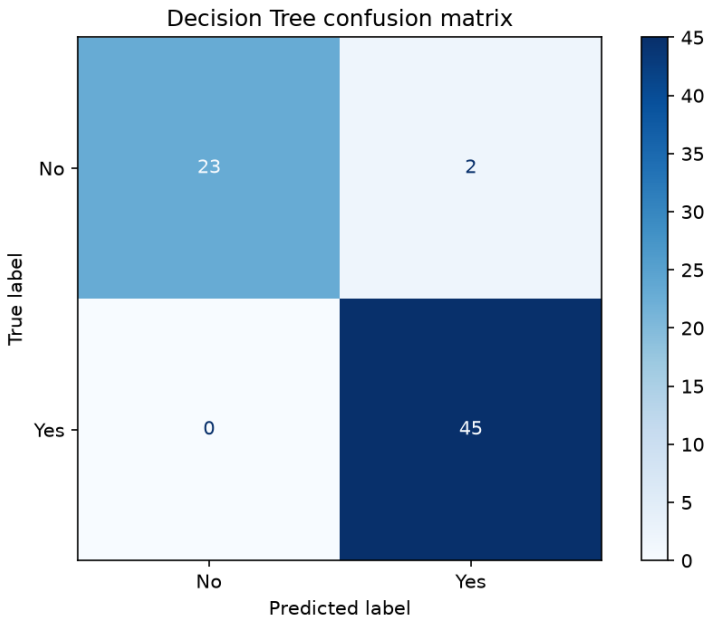


图 7　决策树混淆矩阵

5　失败案例与局限

测试集中只有 2 条错判，因此列出这 2 条错误和 1 条靠近 30 天界限的正确样本。顾客编号 406：距上次购买 33 天，真实标签为 0，两种模型都预测为 1；其累计消费 1160.6、购买 15 件，与近期购买顾客的行为相似。

顾客编号 418：距上次购买 34 天，真实标签为 0，也被预测为 1；其累计消费 1140.6、购买 15 件，出现了类似误判。顾客编号 119 距上次购买 29 天，真实标签为 1；它位于人为设置的 30 天界限附近，虽然本次预测正确，但阈值稍变就可能改变标签。

本实验有三点局限。第一，标签来自“距上次购买的天数”，任务是识别过去 30 天是否购买，不能直接预测未来购买。第二，除顾客编号外有 180 条重复记录；原始数据的随机划分可能让同一条记录的副本进入不同数据集，因此 0.971 只能作为本次划分的结果。第 6.2 节增加了去重后的对照检验。第三，数据仅有 350 条既有顾客记录，没有从未购买的顾客；若用于真实业务，还需收集更广的样本和未来购买标签。

6　独立扩展

6.1　三次随机划分

为检查一次随机划分是否偶然，保持同样的特征、参数搜索范围和 60%/20%/20% 划分方式，额外将随机种子改为 42、21、7 各运行一次。每次仍只用验证集选参数，测试

集不参与调参；主程序截图对应种子 42。

本次扩展只改变数据划分随机种子，没有改动原始 CSV。下表是每个种子的测试集准确率；三次结果的均值和样本标准差另在表下说明。

表 3　原始数据三次随机划分的测试准确率

随机种子	逻辑回归准确率	决策树准确率	说明
42	0.971	0.971	主程序截图
21	0.957	0.957	重新划分
7	0.986	0.986	重新划分

三次测试准确率：逻辑回归与决策树均为 0.971±0.014（均值±样本标准差）。两者在这三个划分下打平，但重复记录问题仍然存在。

6.2　去重后的稳健性检验

原始 350 条记录中，除 Customer ID 外完全相同的记录有 180 条（按“重复的多余行数”计），去重后保留 170 条。为检验重复记录对分数的影响，额外程序 04_去重稳健性检验.py 先去重，再沿用相同特征、参数范围及分层划分比例。每个种子得到训练集 102 条、验证集 34 条、测试集 34 条；仍只用验证集选参数。

表 4　去重后各模型的测试集表现

随机种子	多数类基线准确率	逻辑回归准确率	决策树准确率	决策树召回率
42	0.618	0.941	0.941	1.000
21	0.618	0.941	0.853	0.810
7	0.618	0.941	0.941	1.000

去重后三次测试中，逻辑回归准确率均为 0.941；决策树为 0.941、0.853、0.941，均值约 0.912。种子 21 的决策树漏判 4 条实际为“是”的记录，召回率为 0.810。这比原始随机划分下两模型均约 0.971 的结果更谨慎，也显示两种模型并非始终同分。两组测试集样本数不同，不能把准确率差额全部归因于重复记录；去重后每个测试集只有 34 条，仍需更多独立数据验证。

7　复现步骤

原实验环境：Python 3.11.16；pandas 3.1.0、scikit-learn 1.9.1、matplotlib 3.11.2、numpy 2.4.6。

示例待预测记录为 30 岁、Silver 会员、累计消费 800、购买 10 件、平均评分 4.0、使用折扣；两个模型均将其判为 0。这里的“新”指尚未进入训练集的既有顾客记录，不代表从未购买的顾客。

8 参考文献

[1] scikit-learn developers. LogisticRegression API documentation, version 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html （用于理解 C 参数和逻辑回归模型）。

[2] scikit-learn developers. DecisionTreeClassifier API documentation, version 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html （用于理解 max_depth 和基尼不纯度）。

[3] 实验数据：E-commerce Customer Behavior.csv

附录 A 逐行注释的程序代码

下面列出主程序当前版本的代码，四个分阶段程序与 04_去重稳健性检验.py 均作为独立源码随提交包提供。代码中的注释说明了每一步的用途。

```
from pathlib import Path  # 读取当前脚本所在的文件夹。
import pandas as pd  # 用于读取表格数据。
from sklearn.metrics import accuracy_score, precision_score, recall_score  # 导入评价指标。
from sklearn.model_selection import train_test_split  # 用于划分训练集和测试集。
from sklearn.pipeline import make_pipeline  # 用于把标准化和逻辑回归连起来。
from sklearn.preprocessing import StandardScaler  # 用于统一各列数据的大小范围。
from sklearn.linear_model import LogisticRegression  # 导入逻辑回归模型。
from sklearn.tree import DecisionTreeClassifier  # 导入决策树模型。

folder = Path(__file__).parent  # 找到本程序所在的文件夹。
data = pd.read_csv(folder / "E-commerce Customer Behavior.csv")  # 读取真实数据，原 CSV 不会被修改。
excluded_cols = ["Customer ID", "Gender", "City", "Satisfaction Level", "Days Since Last Purchase"]  # 去掉编号、非行为信息和用来生成答案的列。
X = pd.get_dummies(data.drop(columns=excluded_cols), dtype=int)  # 只在内存中筛选特征，并把会员类型转成数字列。
y = (data["Days Since Last Purchase"] <= 30).astype(int)  # 定义：近 30 天购买过为 1，否则为 0。
print("本实验预测的是近 30 天是否购买；排除的列：", excluded_cols)  # 说明目标和被排除的列。
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)  # 留出 20% 数据作为最终测试集。
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25, random_state=42, stratify=y_dev)  # 其余数据再分成训练集和验证集。

c_values = [0.01, 0.1, 1, 10, 100]  # C 越小，逻辑回归的正则化越强。
lr_val_scores = []  # 保存不同 C 下的验证准确率。
for c in c_values:  # 依次尝试每个 C。
    model = make_pipeline(StandardScaler(), LogisticRegression(C=c, max_iter=1000))  # 先标准化，再训练逻辑回归。
    model.fit(X_train, y_train)  # 用训练集学习。
    lr_val_scores.append(model.score(X_val, y_val))  # 记录验证准确率。

depth_values = [1, 2, 3, 4, 5, 6, 7, 8]  # 尝试八种决策树最大深度。
tree_val_scores = []  # 保存不同深度下的验证准确率。
for depth in depth_values:  # 依次尝试每个深度。
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)  # 创建决策树。
    model.fit(X_train, y_train)  # 用训练集学习。
    tree_val_scores.append(model.score(X_val, y_val))  # 记录验证准确率。

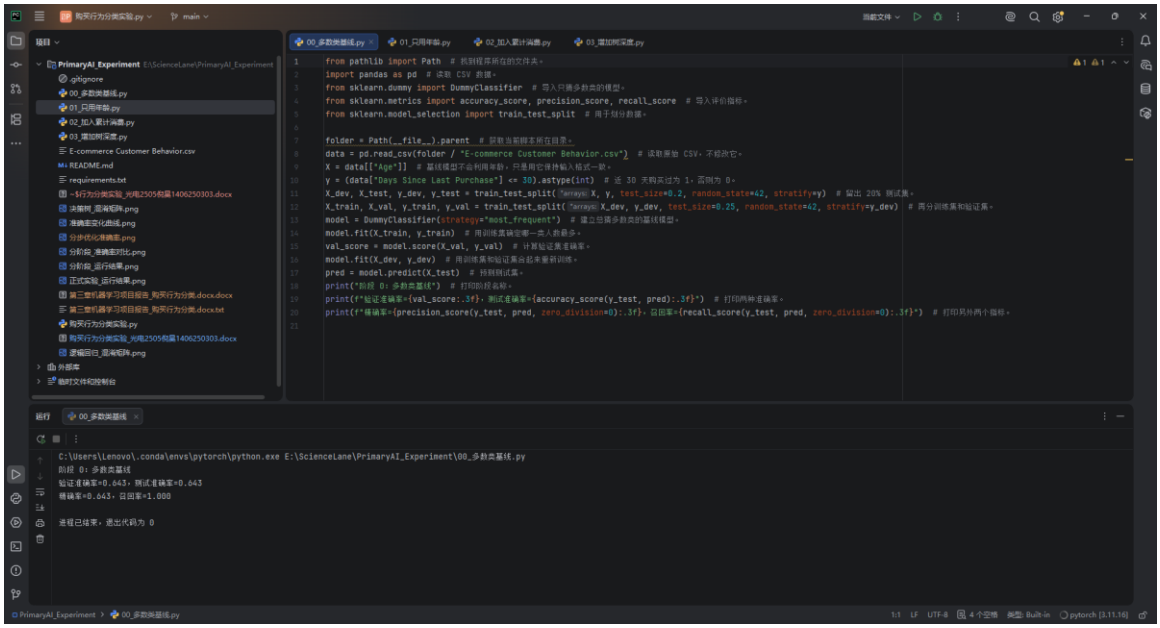
best_c = c_values[lr_val_scores.index(max(lr_val_scores))]  # 选择验证准确率最高的 C。
best_depth = depth_values[tree_val_scores.index(max(tree_val_scores))]  # 选择验证准确率最高的深度。
lr_model = make_pipeline(StandardScaler(), LogisticRegression(C=best_c, max_iter=1000))  # 建立选定参数的逻辑回归。
tree_model = DecisionTreeClassifier(max_depth=best_depth, random_state=42)  # 建立选定深度的决策树。

for name, model in [("逻辑回归", lr_model), ("决策树", tree_model)]:  # 分别评价两种模型。
    model.fit(X_dev, y_dev)  # 用训练集和验证集合起来的数据重新训练。
```

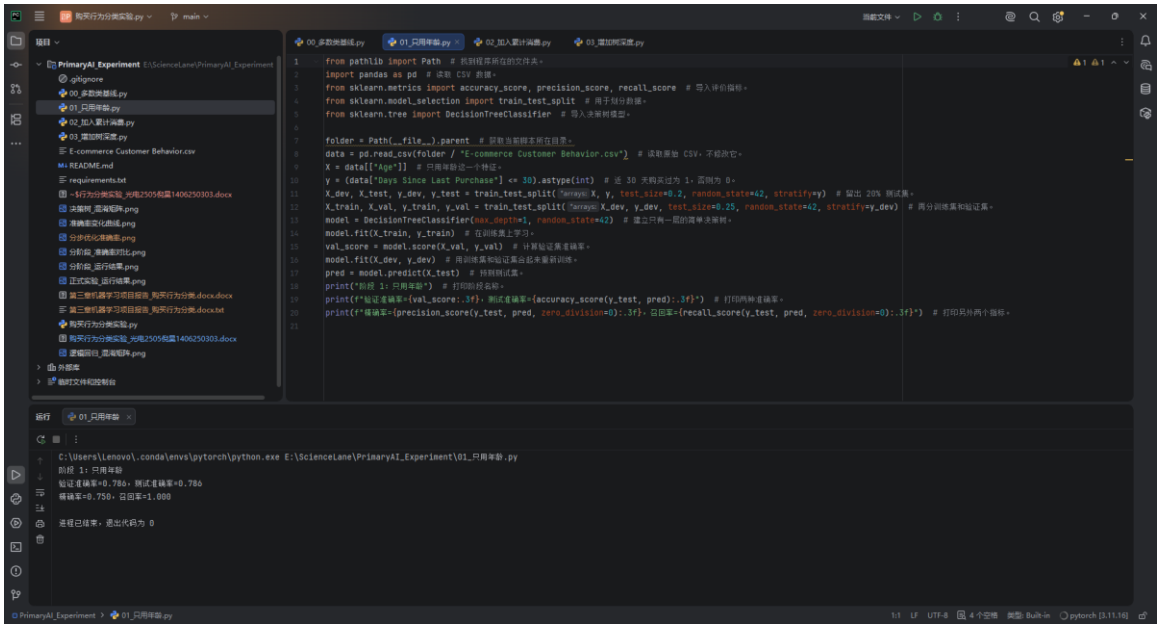
```
40        predictions = model.predict(X_test)  # 预测测试集是否购买。
41        print(f"{name}：准确率={accuracy_score(y_test, predictions):.3f}，精确率
={precision_score(y_test, predictions, zero_division=0):.3f}，召回率={recall_score(y_test,
predictions, zero_division=0):.3f}")  # 打印三个指标。
42
43  new_user = pd.DataFrame([{"Age": 30, "Membership Type": "Silver", "Total Spend": 800,
"Items Purchased": 10, "Average Rating": 4.0, "Discount Applied": True}])  # 准备一位新用户的数
据。
44  new_user = pd.get_dummies(new_user, dtype=int).reindex(columns=X.columns, fill_value=0)
# 让新用户的列与训练数据保持一致。
45  print("新用户预测：逻辑回归=", lr_model.predict(new_user)[0], "决策树=",
tree_model.predict(new_user)[0])  # 输出预测，1 表示购买。
46  print("选用参数：逻辑回归 C=", best_c, "决策树最大深度=", best_depth)  # 输出选用的参数。
```

附录 B　分阶段程序运行截图

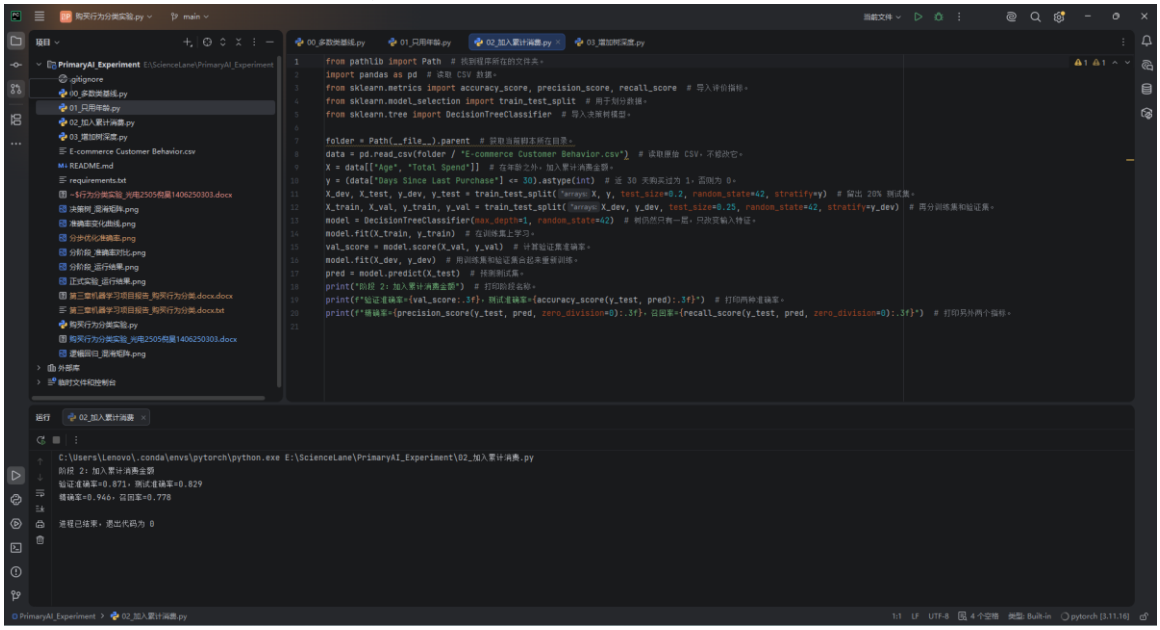
以下截图分别对应表 1 的四个阶段。图片下方的控制台输出与表中数值一致。



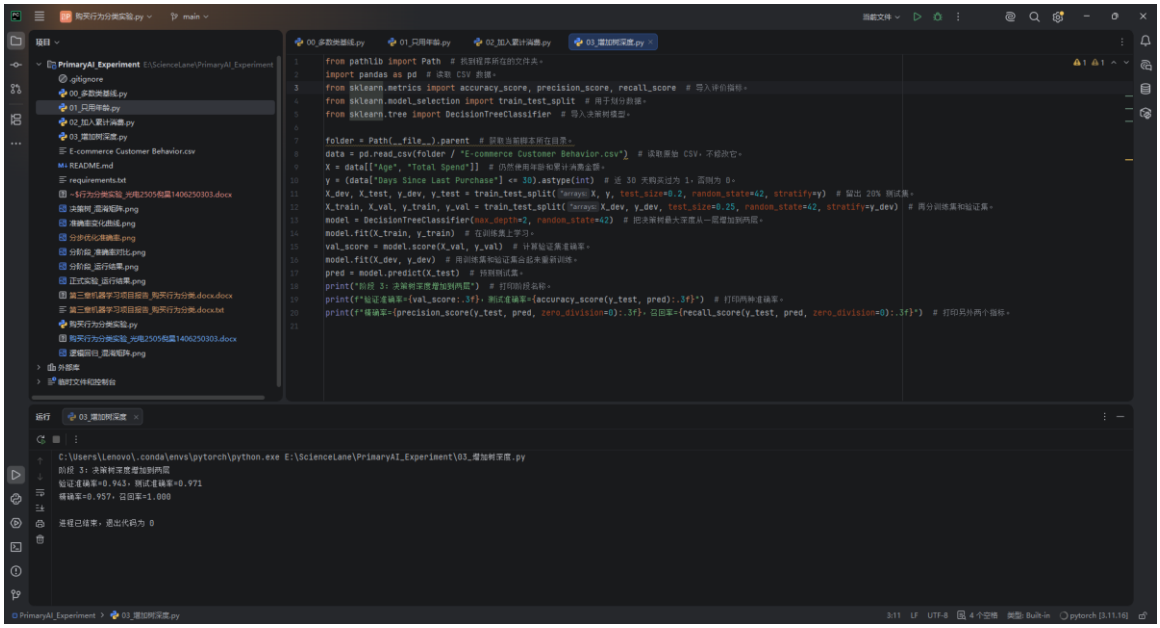
附图 1　阶段 0 程序运行结果



附图 2　阶段 1 程序运行结果



附图 3　阶段 2 程序运行结果



附图 4　阶段 3 程序运行结果